

Sprint Documentation

Sprint 15	Start Date:14/04/26	Final Date:20/04/26
Projetc Name:	Civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Felipe Pacheco Bianchini		
Giovanni da Silva Nascimento		

Sprint Backlog

Task#	Description	Start Date	Final date
79	Tarefa: Validações de Orçamento	14/04/26	20/04/26

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
79	Garantir que o cadastro e manutenção de orçamentos possuam validações completas de integridade, consistência e regras de negócio, evitando dados inválidos e estouro de limite financeiro no sistema.	Felipe Pacheco Bianchini, Giovanni da Silva Nascimento	Done	7	7

Sprint Description

Tarefa: Validações de Orçamento

Objetivo

Garantir que o cadastro e manutenção de orçamentos possuam validações completas de integridade, consistência e regras de negócio, evitando dados inválidos e estouro de limite financeiro no sistema.

Descrição

Implementar validações no backend para os campos da entidade de orçamento, assegurando:

- Dados obrigatórios
- Formatação correta
- Consistência de valores monetários

- Integridade relacional
-

✓ Critérios de Aceite

◆ Remoção de Relações

- Remover o campo IdTipoDespesa de orçamento. Essa relação não é necessária e está incorreta.

◆ Campos Obrigatórios

- - Validar que os campos obrigatórios não estão vazios:
 - anoOrcamento
 - valorOrcamento
 - idInstituicao
-

◆ Validação de Ano do Orçamento

- Deve conter exatamente 4 dígitos

Deve ser um ano válido (ex: ≥ 2000)

Não permitir anos muito distantes do atual (ex: +10 anos no futuro)

Não permitir valores negativos ou zero

◆ Validação de Valor do Orçamento

- Deve ser maior que 0

Não permitir valores negativos

Validar precisão monetária (recomendado usar decimal ao invés de double ⚠)

◆ Validação de Instituição

- idInstituicao deve existir no banco

Não permitir associação com instituição inexistente

Validar integridade da chave estrangeira

◆ Regra de Negócio (Controle de Gastos)

- Validar que a soma das despesas vinculadas (Despesas) não ultrapasse valorOrcamento
- Ao cadastrar ou atualizar despesas:
 - Somar despesas existentes + nova despesa
 - Bloquear operação caso ultrapasse o limite

Retornar erro claro ao usuário informando estouro de orçamento

◆ Validação em Atualização

-
- Ao atualizar valorOrçamento:
 - Não permitir valor menor que a soma das despesas já existentes

Garantir consistência após alteração

◆ Validação de Exclusão

- Não permitir exclusão de orçamento que possua despesas vinculadas
-

◆ Normalização e Boas Práticas

- Garantir uso de tipo monetário adequado (decimal)

Padronizar arredondamento de valores (ex: 2 casas decimais)

Validar entradas antes de persistir no banco

📌 Observações e Sugestões

- Implementar validações na camada **SERVICE**
- Não depender exclusivamente de **DataAnnotations**
- Retornar mensagens claras ao usuário
- Utilizar **transações** ao validar despesas + orçamento para evitar inconsistências
- Os erros retornados ao usuário devem vir em uma lista com todos os erros de uma só vez ao invés de mostrar um erro por vez.
- Verifiquem o trabalho antes do envio

Sprint Results

Relatório - Sprint 15 - Validação de Orçamento

Objetivo da task

Garantir que o cadastro e a manutenção de orçamentos possuam validações completas de integridade, consistência e regras de negócio, evitando dados inválidos e estouro de limite financeiro no sistema. A entrega move as validações para a camada de Service, padroniza o tipo monetário, retorna todos os erros em uma única lista e torna atômicas as operações que dependem do saldo orçamentário.

O que foi produzido

- Remoção do campo `IdTipoDespesa` da entidade `Orcamento` (model, DTO, snapshot, migration).
- Conversão de `ValorOrcamento` de `double` para `decimal` com precisão `18,2` e arredondamento `MidpointRounding.AwayFromZero`.
- Consolidação de todas as regras de validação de orçamento na camada de Service, eliminando validações inline do Controller.
- Criação da exceção `OrcamentoValidationException`, que carrega a lista agregada de erros e é traduzida pelo Controller em `400 Bad Request` com `Data` populado pelos itens inválidos.
- Endpoint `DELETE /api/orcamentos/{id}` com bloqueio da exclusão caso existam despesas vinculadas.
- Regra de negócio no `DespesaService` que bloqueia o cadastro/atualização de despesa quando `soma(ConsumoPrevisto) > ValorOrcamento`.
- Envoltório transacional `IsolationLevel.Serializable` nas operações que dependem do saldo orçamentário (`DespesaService.Create/Update`, `OrcamentoService.Create/Update/RemoveAsync`) para evitar race conditions.
- Atualização do teste `PostDespesa\_WithInactiveFornecedor\_ReturnsBadRequest` para acompanhar a remoção de `IdTipoDespesa`.
- Documentação: `pr-sprint-15-validacao-orcamento.md` e este relatório.

Diagrama de Casos de Uso

```mermaid

flowchart TB

Admin([Administrador Financeiro])

subgraph Orcamento [Orçamento]

UC1[Cadastrar Orçamento]

UC2[Atualizar Orçamento]

UC3[Excluir Orçamento]

end

subgraph Despesa [Despesa]

UC4[Cadastrar Despesa]

UC5[Atualizar Despesa]

end

Admin --> UC1

Admin --> UC2

Admin --> UC3

Admin --> UC4

Admin --> UC5

UC1 -. "«include» valida campos, ano, valor, instituição" .-> V1[[Validação de Orçamento]]

UC2 -. "«include» valida campos + novo valor >= comprometido" .-> V1

UC3 -. "«include» bloqueia se houver despesa vinculada" .-> V2[[Integridade Referencial]]

UC4 -. "«include» sum despesas + nova <= valorOrçamento" .-> V3[[Controle de Estouro]]

UC5 -. "«include» sum despesas - atual + nova <= valorOrçamento" .-> V3

````

Diagrama de Classes Afetadas

```mermaid

classDiagram

```
class Orcamento {
 +int IdOrcamento
 +int AnoOrcamento
 +decimal ValorOrcamento
 +int IdInstituicao
 +Instituicao Instituicao
 +ICollection~Despesa~ Despesas
}
```

```
class OrcamentoDTO {
 +int IdOrcamento
 +int AnoOrcamento
 +decimal ValorOrcamento
 +int IdInstituicao
}
```

```
class OrcamentoService {
 -IOrcamentoRepository _orcamentoRepository
 -IInstituicaoRepository _instituicaoRepository
}
```

```

-AppDbContext _context
+Create(OrcamentoDTO) Task
+Update(OrcamentoDTO, int) Task
+RemoverAsync(int) Task
+ExisteDespesaVinculada(int) Task~bool~
-ValidarOrcamentoAsync(OrcamentoDTO, Orcamento) Task
-ExecuteEmTransacaoAsync(Func~Task~) Task
}

```

```

class OrcamentoController {
+GetAll() IActionResult
+GetOrcamentoById(int) IActionResult
+Post(OrcamentoDTO) IActionResult
+Put(int, OrcamentoDTO) IActionResult
+Delete(int) IActionResult
}

```

```

class IOrcamentoRepository {
+ExisteDespesaVinculada(int) Task~bool~
+SumConsumoByOrcamentoAsync(int) Task~decimal~
}

```

```

class OrcamentoValidationException {
+ICollection~string~ Errors
}

```

```

class DespesaService {
-AppDbContext _context
+Create(DespesaDTO) Task
+Update(DespesaDTO, int) Task
-ValidarResultLimiteOrcamentarioAsync(DespesaDTO, Orcamento) Task
-ExecuteEmTransacaoAsync(Func~Task~) Task
}

```

```
class IDespesaRepository{
 +SumConsumoByOrcamentoAsync(int, int?) Task~decimal~
}
```

OrcamentoController --> OrcamentoService

OrcamentoService --> IOrcamentoRepository

OrcamentoService --> OrcamentoValidationException

DespesaService --> IDespesaRepository

DespesaService --> IOrcamentoRepository

Orcamento --> OrcamentoDTO : mapeado por AutoMapper

...

## ## Diagrama de Sequência — Cadastro de Despesa com Controle de Estouro

```
```mermaid
```

```
sequenceDiagram
```

```
actor Admin as Administrador
```

```
participant Controller as DespesaController
```

```
participant Service as DespesaService
```

```
participant DB as AppDbContext
```

```
participant OrcRepo as IOrcamentoRepository
```

```
participant DespRepo as IDespesaRepository
```

```
Admin->>Controller: POST /api/despesas
```

```
Controller->>Service: Create(despesaDTO)
```

```
Service->>DB: BeginTransactionAsync(Serializable)
```

```
Service->>Service: ValidarCadastroAsync(dto)
```

```
Service->>OrcRepo: GetById(IdOrcamento)
```

```
OrcRepo-->>Service: Orcamento
```

```
Service->>DespRepo: SumConsumoByOrcamentoAsync(IdOrcamento, ignoreId)
```

```
DespRepo-->>Service: totalExistente
```

```
alt totalExistente + novoValor > ValorOrcamento
```

```
    Service-->>Controller: ArgumentException (estouro)
```

```
    Service->>DB: RollbackAsync()
```

```

    Controller-->>Admin: 400 BadRequest + mensagem
else dentro do limite
    Service->>DespRepo: Add(despesa)
    Service->>DB: CommitAsync()
    Service-->>Controller: OK
    Controller-->>Admin: 200 OK + DTO
end
` ` `

```

Diagrama de Sequência — Atualização de Orçamento com Lista de Erros

```

` ` ` mermaid
sequenceDiagram
    actor Admin as Administrador
    participant Controller as OrcamentoController
    participant Service as OrcamentoService
    participant DB as AppDbContext
    participant OrcRepo as IOrcamentoRepository
    participant InstRepo as IIstituicaoRepository

    Admin->>Controller: PUT /api/orcamentos/{id}
    Controller->>Service: Update(dto, id)
    Service->>DB: BeginTransactionAsync(Serializable)
    Service->>OrcRepo: GetById(id)
    OrcRepo-->>Service: Orcamento existente
    Service->>Service: ValidarOrcamentoAsync(dto, existente)
    Service->>InstRepo: GetById(IdInstituicao)
    InstRepo-->>Service: Instituicao
    Service->>OrcRepo: SumConsumoByOrcamentoAsync(id)
    OrcRepo-->>Service: totalComprometido
    alt errors.Count > 0
        Service-->>Controller: OrcamentoValidationException(errors)
        Service->>DB: RollbackAsync()
        Controller-->>Admin: 400 BadRequest + Data=[erros]
    end

```

```

else sem erros

    Service->>OrcRepo: Update(orcamento)

    Service->>DB: CommitAsync()

    Service-->>Controller: OK

    Controller-->>Admin: 200 OK + DTO

end
` ` `

```

Diagrama de Sequência — Exclusão Protegida de Orçamento

```

` ` ` mermaid
sequenceDiagram
    actor Admin as Administrador
    participant Controller as OrcamentoController
    participant Service as OrcamentoService
    participant DB as AppDbContext
    participant OrcRepo as IOrcamentoRepository

    Admin->>Controller: DELETE /api/orcamentos/{id}
    Controller->>Service: RemoverAsync(id)
    Service->>DB: BeginTransactionAsync(Serializable)
    Service->>OrcRepo: GetById(id)
    alt orçamento inexistente
        Service-->>Controller: KeyNotFoundException
        Controller-->>Admin: 404 NotFound
    else existe
        Service->>OrcRepo: ExisteDespesaVinculada(id)
        alt possui despesas
            Service-->>Controller: OrcamentoValidationException
            Service->>DB: RollbackAsync()
            Controller-->>Admin: 400 BadRequest + Data=[erro]
        else sem despesas
            Service->>OrcRepo: Remove(orcamento)
            Service->>DB: CommitAsync()
        end
    end
end

```

Controller-->>Admin: 200 OK

end

end

...

Telas produzidas

Não há telas nesta entrega. A task é exclusivamente de backend (validações, regras de negócio e endpoints REST). A verificação manual em clientes HTTP (Swagger UI e chamadas diretas) não gerou artefatos visuais de UI.

Testes realizados

Testes automatizados

- Execução completa da suíte existente com `DOTNET\_ROLL\_FORWARD=LatestMajor dotnet test Civitas.WebAPI.Tests/Civitas.WebAPI.Tests.csproj`.
- Resultado: ****33 testes passando / 5 falhando****.
- As 5 falhas foram reproduzidas ****antes**** das mudanças (via `git stash`), confirmando que são ****pré-existent**** e não foram introduzidas por este PR:
 - `CepEndpointsTests.GetCep\_WithoutToken\_ReturnsUnauthorized`
 - `AuthorizationEndpointsTests.GetProtectedRoute\_WithoutToken\_Returns401\_WhenDevFalse`
 - `AuthorizationEndpointsTests.GetProtectedRoute\_WithInvalidToken\_Returns401\_WhenDevFalse`
 - `AuthorizationEndpointsTests.GetProtectedRoute\_WithExpiredToken\_Returns401\_WhenDevFalse`
 - `FornecedorValidationEndpointsTests.PostDespesa\_WithInactiveFornecedor\_ReturnsBadRequest`
- Ajuste pontual no seed do teste `PostDespesa\_WithInactiveFornecedor\_ReturnsBadRequest` para remover `IdTipoDespesa`, que não existe mais no modelo.

Verificações de build e migração

- `dotnet build Civitas.WebAPI/Civitas.WebAPI.sln` — build limpo, 0 erros, warnings idênticos à baseline anterior ao PR.
- `dotnet ef migrations add OrcamentoValidacoesRefactor` — migration gerada, revisada, conferida como alinhada ao modelo (remove `idtipodespesa`, shadow FK `TipoDespesaId` e altera `valororcamento` para `numeric(18,2)`).

Testes manuais

- Swagger UI com tentativas de cadastro e atualização de orçamento usando:
 - ``ValorOrçamento = 0`` — retorna 400 com ``ValorOrçamento` deve ser maior que zero``.
 - ``AnoOrçamento = 1999`` — retorna 400 com ``AnoOrçamento` deve ser maior ou igual a 2000``.
 - ``AnoOrçamento = (ano atual + 15)`` — retorna 400 com mensagem de limite de anos à frente.
 - ``IdInstituicao`` inexistente — retorna 400 com ``A instituicao informada nao foi encontrada``.
 - Atualização de ``ValorOrçamento`` para valor inferior à soma das despesas — retorna 400 com mensagem descritiva dos dois valores.
- Verificação de estouro via ``POST /api/despesas`` com valor que ultrapassa o teto — retorno com mensagem contendo excedente, total comprometido e teto.
- ``DELETE /api/orcamentos/{id}`` com despesa vinculada — retorna 400 com ``Nao e possivel excluir um orcamento que possui despesas vinculadas``.

O que foi bem — Aprendizados

- A centralização da validação no Service seguiu o mesmo padrão já estabelecido por ``UsuarioService`` / ``UsuarioValidationException``, reduzindo atrito de revisão e facilitando o onboarding de quem já conhecia esse padrão.
- A descoberta de que o repositório já somava ``ConsumoPrevisto`` como proxy do valor comprometido (em ``InstituicaoRepository`` e ``SecretariaRepository``) evitou a introdução de um campo monetário novo em ``Despesa``, mantendo as mudanças cirúrgicas e alinhadas com a regra já praticada.
- O uso de ``IsolationLevel.Serializable`` nas transações eliminou uma race condition real no controle de saldo (leitura da soma + inserção da despesa), reforçando a integridade financeira sob concorrência — aprendizado diretamente aplicável a futuras regras monetárias do sistema.
- O ``MidpointRounding.AwayFromZero`` combinado com ``HasPrecision(18,2)`` padroniza o tratamento monetário e evita o clássico “rounding bug” que o tipo ``double`` produziria em somas financeiras.
- A migração do modelo snapshot evidenciou um drift pré-existente (``despesa.situacao`` → ``status``) originado em um PR anterior; a correção foi incorporada sem custo adicional, deixando o snapshot íntegro.
- O agrupamento dos erros em uma única lista melhora significativamente a experiência do usuário final (todos os erros em uma resposta) e reduz idas e vindas na tela de cadastro.

O que não pôde ser implementado — Justificativas

- ****Campo monetário próprio em ``Despesa``.** A spec cita “soma das despesas vinculadas” sem fornecer um ``ValorDespesa``. Como ``Despesa`` não possui esse campo, o cálculo usa ``ConsumoPrevisto``, seguindo o padrão já praticado pelo restante do sistema. A criação de um ``ValorDespesa`` dedicado é uma mudança de domínio maior e depende de alinhamento com o time de negócio; foi documentada como ponto de extensão nas observações do PR.
- ****Lista de erros agregada também no ``DespesaService``.** O ``DespesaService`` mantém o padrão existente de lançar ``ArgumentException`` individualmente (uma regra por vez). A spec de lista agregada está no contexto de Orçamento; ampliar esse comportamento para ``Despesa`` exigiria refatorar todas as

regras de validação de despesa, o que extrapola o escopo do card. Pode ser endereçado em uma task dedicada à uniformização de erros.

- ****Testes automatizados específicos para as novas regras.**** O tempo da sprint foi priorizado na implementação das regras, migração e transações. A validação foi feita via build limpo, suíte existente rodada e testes manuais no Swagger. Testes de unidade e de integração específicos para as novas validações (ano, valor, estouro, delete bloqueado) ficam como pendência direta para a próxima sprint.

- ****Correção dos testes pré-existentes.**** Os 5 testes que já falhavam não foram corrigidos nesta task por estarem fora do escopo definido pelo card de Validação de Orçamento. Recomenda-se task específica de manutenção da suíte de testes.

- ****Migration versionada no repositório.**** O `.gitignore` do projeto ignora `**/Migrations/`, então apenas o `AppDbContextModelSnapshot.cs` foi atualizado e commitado. A migration física (`OrcamentoValidacoesRefactor.cs`) permanece local por política do repositório.

Verificações executadas

- `dotnet build Civitas.WebAPI/Civitas.WebAPI.sln`
- `dotnet ef migrations add OrcamentoValidacoesRefactor`
- `DOTNET_ROLL_FORWARD=LatestMajor dotnet test Civitas.WebAPI.Tests/Civitas.WebAPI.Tests.csproj`
- `git stash` + re-teste para confirmação de falhas pré-existentes

Observações

- A policy do repositório de ignorar migrations coloca o snapshot como a fonte de verdade do modelo. Qualquer dev que subir o branch deve rodar `dotnet ef database update` localmente para aplicar a nova migration gerada.
- O ambiente local exigiu `DOTNET_ROLL_FORWARD=LatestMajor` pois a máquina não possui runtime .NET 9 instalado (apenas .NET 8 e .NET 10). O projeto continua tendo como alvo `net9.0`.
- A alteração de `InstituicaoOrcamentoDisponivelDTO.TotalOrcamentoDisponivel` para `decimal` é um ajuste de consistência com `SecretariaOrcamentoDisponivelDTO` (que já era `decimal`) e preserva precisão monetária.

Assinaturas